C01AT2

1. Assigning doctors to shifts using Backtracking Search

Let the variables be the shifts:

M = Morning

A = Afternoon

N = Night

Each shift must be assigned one doctor from D1, D2, D3. Each doctor must be assigned exactly once.

Constraints:

1. D1 cannot work Night.
2. D2 must work before D3.
3. Only one doctor can work each shift.
4. D3 cannot work Morning.
5. All doctors must be assigned exactly one shift.

Because each doctor has one shift and the order is Morning < Afternoon < Night, "D2 must work before D3" means D2 must be in an earlier shift than D3.

Backtracking steps and constraint checks

Start by assigning Morning, then Afternoon, then Night. At each step, reject any assignment that violates a constraint.

1. Try M = D1. This is allowed: D1 is not assigned Night.
2. Try A = D2. This is allowed: D2 is before D3, who has not yet been assigned.
3. Try N = D3. This is allowed: D3 is not in Morning, D2 is before D3, and each doctor is assigned once.
4. All shifts have one doctor, all doctors are used exactly once, and every constraint is satisfied. The search stops.

Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

This assignment satisfies all the stated constraints.

2. Grid search from S to G

C01AT2

Important information missing from the assessment

The text says the environment is a 5×5 grid with obstacles, but the PDF does not provide the grid layout or the coordinates of S, G, or the obstacles. Without those details, no unique node-expansion sequence or optimal path can be calculated.

Search approach

The movement cost is 1 for every move, so Breadth-First Search (BFS) finds an optimal path, provided the robot knows the grid. BFS uses a FIFO queue: it expands nodes in the order they were discovered.

The Manhattan distance from a cell (r, c) to goal (rg, cg) is:

$$h(r, c) = |r - rg| + |c - cg|$$

Manhattan distance is a heuristic commonly used with A* search. A* keeps a priority queue ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost so far. BFS does not use a heuristic to order its queue, so the question appears to combine BFS and A* instructions. For a BFS answer, use the FIFO queue; if the instructor expects heuristic-based priority queue updates, use A* instead.

BFS procedure

1. Put S in the queue and mark it visited.
2. Remove the front cell from the queue and examine its valid neighbours: Up, Down, Left, Right.
3. Reject any neighbour outside the grid, blocked by an obstacle, or already visited.
4. Add each remaining neighbour to the back of the queue and record its parent.
5. Repeat until G is removed from the queue.
6. Follow the recorded parent links backward from G to reconstruct the path.
7. Since each move costs 1, the path cost is the number of moves.

Pseudocode

```
function BFS(grid, S, G):
    queue ← [S]
    visited ← {S}
    parent[S] ← none

    while queue is not empty:
        current ← remove_front(queue)

        if current = G:
            return ReconstructPath(parent, G)

        for each neighbour in [Up, Down, Left, Right]:
            if neighbour is inside grid
               and neighbour is not an obstacle
               and neighbour is not in visited:
```

```
            visited.add(neighbour)
            parent[neighbour] ←current
            queue.add_back(neighbour)

    return "No path exists"
```

Conclusion: BFS returns a least-move path when every move costs 1. The exact queue updates, path, and total cost require the missing grid diagram or coordinates.

3. Autonomous rescue robot

a) How the constraints affect decision-making

The robot must plan movement while accounting for its limited information and the different risks of possible routes.

Four-direction movement: The robot can move only up, down, left, or right, which limits its available actions at every cell.

Blocked cells: The robot must avoid obstacles and cannot plan a route through them.

Limited sensing: It can observe only adjacent cells, so it does not initially know the whole map. It must update its map as it moves.

Movement cost of 1: Every ordinary move adds 1 to the total route cost.

Risky-zone cost of 2 extra: Entering a risky cell costs $1 + 2 = 3$. A route with fewer steps may therefore be more expensive than a longer, safer route.

Minimum total cost and safety: The robot should compare route costs while avoiding known blocked cells and giving preference to safer routes when costs differ.

Online search: The robot must act, gather new percepts, update its knowledge, and replan as needed. It cannot rely entirely on a complete map in advance.

The robot's accumulated route cost can be expressed as:

Total cost = number of ordinary movement steps + 2 × number of risky cells entered

This assumes the extra risky-zone cost is charged in addition to the standard movement cost.

b) Suitable solution approach: Online search with Uniform Cost Search

Use an online search agent that learns the map through local sensing, and Uniform Cost Search (UCS) to select the least-cost route through the known safe and traversable cells.

1. Begin at S and initialise the known map with the start cell.
2. Sense the four adjacent cells.
3. Mark newly observed cells as open, blocked, or risky.

4. For each reachable known cell, calculate the cost of entering it:
   - Ordinary cell: 1
   - Risky cell: 3
   - Blocked cell: not traversable
5. Use UCS to expand the known route with the lowest accumulated cost.
6. Move one step along the selected route.
7. Sense the new surroundings and update the map.
8. If a newly discovered obstacle or risk changes the route, run UCS again using the updated map.
9. Continue until the robot reaches G; report the path and its total cost.

UCS pseudocode:

```
function OnlineRescueAgent(S, G):
  knownMap ← map containing only observed information
  current ← S
  totalCost ← 0

  while current ≠ G:
    observations ← SenseAdjacentCells(current)
    Update(knownMap, observations)

    path ← UniformCostSearch(
          knownMap,
          current,
          G,
          entryCost = 1 for ordinary cells,
                3 for risky cells,
          blocked cells excluded
        )

    if path does not exist:
      return "No known safe path; continue sensing or request assistance"

    nextCell ← first step of path
    MoveTo(nextCell)
    totalCost ← totalCost + EntryCost(nextCell)

    current ← nextCell

  return "Survivor reached", totalCost
```

UCS uses a priority queue ordered by accumulated path cost. When it removes the goal as the lowest-cost node, it has found a least-cost route through the currently known map . If the robot later discovers new information, it may need to replan.

c) AI concepts used

Intelligent agent: The robot receives percepts from sensors, maintains information about its surroundings, chooses an action, and acts to reach the

survivor. It observes and updates its knowledge throughout the mission.

Rationality: A rational robot chooses the action that best supports its performance measure, given the information it has at that moment. Here, it should seek the survivor while minimising movement and risk costs and avoiding known obstacles. Rationality does not mean it always knows the globally best route; its decisions depend on its limited percepts.

Environment type:

Partially observable: The robot sees only adjacent cells, not the whole building.

Sequential: Each movement affects later choices and the cost accumulated so far.

Dynamic or changing from the robot's perspective: The map changes as the robot discovers more cells; conditions may also change during rescue.

Stochastic or uncertain: The robot may face uncertainty in sensing or in the actual condition of a route.

Single-agent for planning purposes: The robot is the decision-making agent, although survivors or other responders may affect the surroundings.

Unknown: The robot does not initially know every obstacle or risky area.

d) Why this approach is suitable

UCS is appropriate because the route costs are not all equal : ordinary moves cost 1, while entering a risky zone costs 3. BFS treats every step equally and would find the route with the fewest moves, which may pass through costly risky areas. UCS accounts for the actual accumulated cost and selects the least-cost known route.

The online-search component is necessary because the robot can sense only adjacent cells. It discovers the building progressively, updates its map, and replans when it finds new obstacles or risks. Together, online search and UCS support decisions based on both route cost and changing knowledge. The robot should also avoid known dangerous or blocked cells to meet the safety requirement.

If you upload the grid for Question 2, I can add its exact node expansion, queue updates, optimal path, and cost.